

What is Apache Airflow?

Apache Airflow is an open-source workflow orchestration tool used to automate, schedule, and monitor data pipelines and workflows. It was developed by Airbnb and is now managed by the Apache Software Foundation.

Airflow is mainly used in data engineering for ETL processes, machine learning workflows, cloud automation, and batch data processing.

Airflow allows users to define workflows using Python programming. These workflows are called DAGs, where tasks are arranged in a sequence based on dependencies.

Airflow automatically schedules and executes tasks at specified intervals such as hourly, daily, or weekly.

One of the major features of Airflow is its web interface, which helps users monitor workflows, check task execution status, retry failed jobs, and view logs. Airflow also supports integration with tools such as Spark, Hadoop, Hive, AWS, Azure, Google Cloud, Snowflake, and Databricks.

Airflow follows the concept of “workflow as code,” making workflows flexible, reusable, and easy to maintain.

Due to its scalability and monitoring capabilities, it is widely used in modern data engineering environments.

Advantages of Airflow

Apache Airflow provides several advantages in workflow management and data engineering.

One important advantage is automation. Airflow automates repetitive tasks and reduces manual intervention in data processing.

Another advantage is scalability. Airflow can handle both small and large workflows efficiently using different executors and distributed systems.

It supports parallel execution of tasks, which improves performance.

Airflow is highly flexible because workflows are written in Python. Developers can customize workflows according to business requirements and integrate them with external systems easily.

The monitoring capability of Airflow is also very useful.

The web interface allows users to monitor workflows, track execution history, check logs, and retry failed tasks. This improves reliability and troubleshooting.

Airflow supports task dependency management, ensuring tasks run in the correct order.

Scheduling features allow workflows to run automatically at specified times.

Another advantage is integration support. Airflow can connect with databases, cloud platforms, APIs, Hadoop, Spark, and many other technologies used in data engineering.

Since Airflow is open source, it has strong community support, regular updates, and no licensing cost. This makes it cost-effective for organizations.

Airflow Architecture

Apache Airflow architecture consists of multiple components that work together to execute workflows.

The main components are the Web Server, Scheduler, Metadata Database, Executor, Worker Nodes, and DAG files.

The Web Server provides the graphical user interface where users can monitor workflows, trigger DAGs, view logs, and manage tasks.

The Scheduler is the core component responsible for checking DAG files and scheduling tasks based on dependencies and execution time.

It continuously monitors workflows and sends tasks for execution.

The Metadata Database stores information related to DAGs, task status, execution history, scheduling details, and logs.

Databases such as PostgreSQL and MySQL are commonly used.

The Executor decides how tasks should run. It manages task execution and distributes tasks to worker nodes.

Different executors provide different levels of scalability and parallelism.

Worker Nodes execute the actual tasks defined in workflows. These tasks may include SQL execution, Spark jobs, API calls, or file processing.

The DAG folder contains Python scripts defining workflows and dependencies. Airflow continuously scans this folder to identify new or modified DAGs.

All these components work together to provide workflow orchestration, scheduling, monitoring, and execution in a scalable environment.

What is a DAG and What are Executors?

A DAG in Apache Airflow stands for Directed Acyclic Graph.

It represents a workflow where tasks are connected in a sequence based on dependencies. “Directed” means tasks move in a defined direction, and “Acyclic” means there are no loops in the workflow.

Each DAG contains multiple tasks. A task can perform operations such as data extraction, transformation, loading, report generation, or sending notifications. Dependencies ensure tasks execute in the correct order.

DAGs are written using Python code and stored in the DAG folder. The scheduler reads DAG files and schedules tasks automatically. DAGs help organize workflows clearly and efficiently.

Executors are components responsible for running tasks in Airflow. They determine where and how tasks are executed.

The Sequential Executor runs one task at a time and is mainly used for testing. The Local Executor supports parallel task execution on a single machine.

The Celery Executor distributes tasks across multiple worker machines using Celery, making it suitable for large-scale environments.

The Kubernetes Executor runs tasks inside Kubernetes pods, providing scalability, isolation, and cloud-native execution.

Executors improve performance, scalability, and resource management in Airflow workflows.